

Tarea 3: Procesamiento de Imágenes en CUDA

Matriz de Covarianza

Benjamín Jimenez, Juan Umaña, Javier Torres
Ingeniería Civil Informática
Universidad de Concepción (UdeC)

Resumen—Este informe presenta el cálculo de la matriz de covarianza para el dataset de imágenes DIV2K utilizando procesamiento en GPU. Se compara un enfoque tradicional de cálculo frente a uno asíncrono implementado con CUDA Streams. Este último busca evadir los límites físicos de la VRAM al procesar grandes volúmenes de imágenes aplanadas, demostrando cómo la orquestación asíncrona permite mitigar restricciones de memoria y mejorar la eficiencia general del sistema mediante el solapamiento de copias y cómputo.

I. INTRODUCCIÓN

El cálculo de la matriz de covarianza es fundamental en múltiples algoritmos de procesamiento de imágenes y aprendizaje automático, requiriendo operaciones matriciales intensivas. Para un conjunto de imágenes aplanadas, el problema matemático implica inicialmente calcular el vector promedio de las características, seguido del centrado de los datos y, finalmente, la obtención de la matriz de covarianza mediante el producto punto de la matriz de datos centrados por su transpuesta.

A medida que el tamaño o la resolución de las imágenes aumenta, el volumen de datos aplanados crece rápidamente. Esto impone fuertes restricciones de memoria en dispositivos GPU modernos, los cuales poseen una capacidad de Video RAM (VRAM) limitada. En particular, intentar cargar el dataset completo en la memoria del dispositivo suele resultar en errores, haciendo necesario el diseño de estrategias de particionamiento y manejo asíncrono de memoria.

II. ENTORNO DE EJECUCIÓN

El desarrollo y experimentación de este proyecto se llevó a cabo en un sistema operativo Linux. La aceleración por hardware se realizó utilizando una unidad de procesamiento gráfico (GPU) NVIDIA GeForce GTX 1650 Ti, la cual cuenta con aproximadamente 3715 MB de VRAM disponible.

Especificaciones del Host:

- CPU:
- Memoria RAM:

III. DECISIONES DE IMPLEMENTACIÓN ALGORÍTMICA

III-A. Enfoque Tradicional (Experimento 1)

En el enfoque tradicional, la ejecución se realiza utilizando el Stream 0 (por defecto) de CUDA. Las transferencias de datos entre el host y el dispositivo operan mediante copias sincrónicas (`cudaMemcpy`). Para el cálculo de la covarianza se implementó una estrategia de memoria compartida o *Tiling*,

la cual mejora los accesos a memoria global al reutilizar bloques de datos dentro de los *Thread Blocks*.

Sin embargo, debido a que toda la memoria requerida debe reservarse en una sola pasada de forma estática en la VRAM, este enfoque sufre rápidamente de errores de *Out of Memory* (OOM). Específicamente, al intentar escalar la resolución de las imágenes a 112×84 píxeles, los 3715 MB de VRAM de la GTX 1650 Ti resultan insuficientes para alojar las matrices temporales y finales, causando la interrupción prematura del programa.

III-B. Orquestación con CUDA Streams (Experimento 2)

Para superar la limitante de memoria y mejorar el rendimiento, se diseñó una estrategia asíncrona basada en CUDA Streams. Esta aproximación consiste en procesar los datos particionados en múltiples lotes (*batches*).

Se utilizó memoria paginada bloqueada (*Pinned Memory*) en el host, asignada mediante `cudaMallocHost`, lo cual permite un acceso directo a la memoria (DMA) por parte del driver de NVIDIA. Las transferencias entre CPU y GPU se ejecutaron utilizando la instrucción `cudaMemcpyAsync`. De este modo, fue posible orquestar una técnica de *latency hiding* o solapamiento, donde se ocultan los tiempos de transferencia PCI-Express al realizar cómputo en la GPU de manera simultánea al envío y recepción del siguiente y anterior lote de datos.

IV. EVALUACIÓN EXPERIMENTAL

En esta sección se evalúa el rendimiento del algoritmo particionado. Se analizan distintos números de streams $S \in \{1, 2, 4, 8, 16\}$ para la resolución de 112×84 .

Tabla I
TIEMPOS DE EJECUCIÓN (MS) PARA DISTINTAS CONFIGURACIONES DE STREAMS

Streams (S)	Copia a GPU	Cómputo	Copia a Host	Tiempo Total
1	2	4	8	16

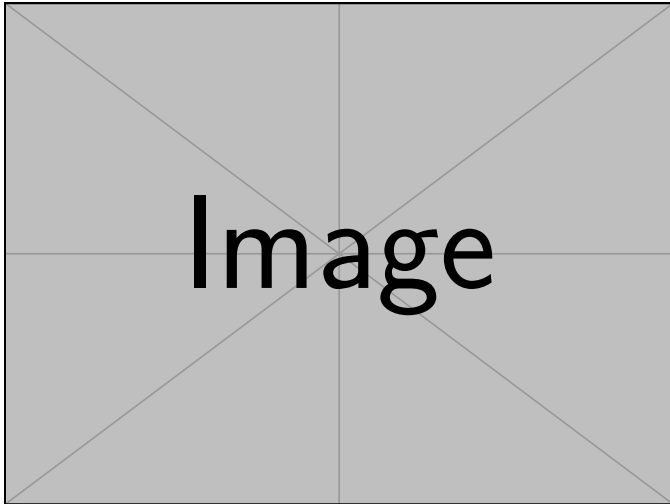


Figura 1. Gráfico de Speedup en función de la cantidad de streams utilizados.

Para visualizar de manera más concreta el solapamiento, se utilizó la herramienta de perfilado Nsight Systems (*nsys*).

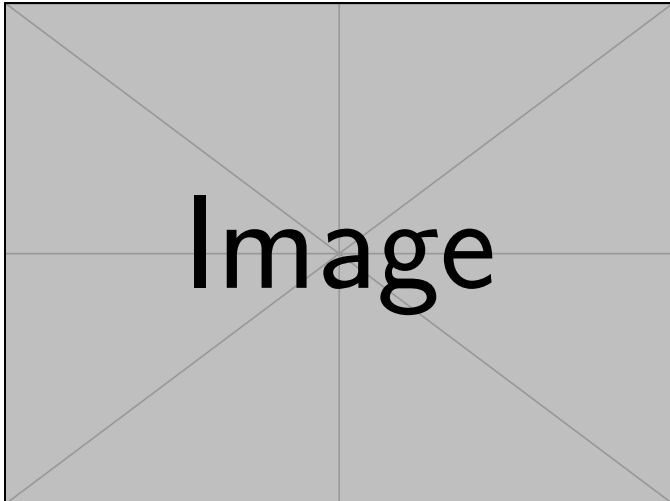


Figura 2. Perfilado en Nsight Systems demostrando el solapamiento entre la transferencia de datos del Host y el cómputo en la GPU.

V. CONCLUSIÓN Y ANÁLISIS CRÍTICO

El desarrollo e implementación de algoritmos en paralelo mediante CUDA requiere de un análisis detallado no solo del problema matemático, sino del hardware subyacente. Los resultados obtenidos muestran que al implementar múltiples streams de ejecución simultáneos se consigue ocultar la latencia de las transferencias a través del bus PCIe.

Se observó que, a partir de cierta cantidad de streams (ver Tabla ??), la mejora en el rendimiento deja de ser significativa. Esto ocurre debido a que la naturaleza de este algoritmo es altamente dependiente del cómputo (*compute-bound*), llegando a ocupar aproximadamente el 99 % del tiempo de ejecución total en operaciones matemáticas dentro de la GPU. Una vez que las latencias de memoria del bus PCIe han sido completamente solapadas (*latency hiding*), la ganancia teórica

se estanca y el tiempo total queda dictado estrictamente por la velocidad del reloj de los núcleos CUDA.

Sin embargo, el mayor logro de la orquestación de streams en este proyecto no fue solo el solapamiento de transferencias, sino su uso como técnica de viabilidad para procesamiento *out-of-core*. El enfoque asíncrono demostró ser una solución exitosa para resolver el cálculo de la covarianza de imágenes a una resolución escalada de 112×84 sobre un conjunto de datos que superaba la memoria física instalada (3715 MB de VRAM). De esta forma, el fraccionamiento de la carga permitió evadir los límites del dispositivo sin comprometer el resultado matemático final.